Learning Taylor Series Expansions via Sequence-to-Sequence Models: An LSTM and Transformer Study

Darisi Surya Teja
Mathematics and Computing
Indian Institute of Technology, Dharwad
mc23bt012@iitdh.ac.in

***Abstract*—Computing the Taylor series expansion of a mathematical function is a fundamental operation in numerical analysis and scientific computing. Classical symbolic algebra systems perform this computation exactly through symbolic differentiation, but their cost scales with function complexity and they operate serially on a single function at a time. In this work, we frame Taylor expansion as a sequence-to-sequence learning problem and train two neural architectures – a bidirectional LSTM encoder-decoder and a Transformer encoder-decoder – to predict fourth-order Taylor expansions for a broad class of mathematical functions around multiple expansion points. Both input functions and their expansions are represented in prefix (Polish) notation using a custom tokenizer supporting rational coefficients. A dataset of 12,000 labelled examples is generated using SymPy, covering polynomials, trigonometric, exponential, logarithmic, and composite functions expanded around $a \in \{0, 1/2, 1\}$. Experimental results show the Transformer achieves 97.2% sequence accuracy and 84.25 BLEU-4, outperforming the LSTM across all three evaluation metrics while using approximately five times fewer parameters. A timing benchmark against SymPy demonstrates that the Transformer provides a 2.1× speedup for very complex functions and a 2.3× batch speedup when processing 16 functions simultaneously, without compromising prediction accuracy.**

***Index Terms*—Taylor series, sequence-to-sequence learning, Transformer, LSTM, symbolic mathematics, prefix notation, neural program synthesis**

I. Introduction

The Taylor series is among the most widely applied tools in mathematics. Given a smooth function f and an expansion point a, its n-th order expansion is

$$f(x) \approx \sum_{k=0}^{n} \frac{f^{(k)}(a)}{k!}(x-a)^k. \tag{1}$$

It arises in numerical analysis for approximating transcendental functions, in physics for linearising equations of motion, in control theory for local system identification, and in statistics for moment calculations. Symbolic algebra systems such as SymPy [1] compute this exactly through term-by-term differentiation. However, three practical constraints limit their applicability at scale.

First, symbolic systems operate *serially*. Computing the Taylor expansion of a million distinct functions requires a million sequential engine calls – there is no native batch parallelism. Second, the computational cost of symbolic differentiation scales with expression depth: a single call on a deeply nested composite such as $\exp(\sin(x^2+\cos(x)))$ can take 80–175 milliseconds. Third, and most critically, symbolic engines fail entirely when the function is not available in closed form, such as when it is defined by a numerical ODE, a black-box simulator, or a learned neural representation. In all such cases, the standard toolchain provides no answer.

Neural sequence-to-sequence (seq2seq) models offer a complementary paradigm. Rather than computing the expansion algorithmically, they learn to predict it from data. Once trained, the model runs in fixed time on GPU regardless of function complexity, processes entire batches of functions in parallel, and can in principle generalise to function classes outside its training distribution.

This work makes the following contributions.

- We construct a principled dataset of 12,000 labelled (function, Taylor expansion) pairs using SymPy, spanning five function families and three expansion points, with both input and output serialised in prefix notation using a custom tokenizer supporting rational coefficients.
- We implement and train two seq2seq architectures under identical conditions: a bidirectional LSTM encoder-decoder with 3.6M parameters and a Transformer encoder-decoder with only 693K parameters.
- We report token accuracy, sequence accuracy, and BLEU-4 on a held-out test set of 1,000 examples.
- We benchmark inference latency against SymPy across four function complexity categories on an NVIDIA RTX 3050 GPU and demonstrate conditions under which the neural approach provides a practical throughput advantage.

II. Motivation: Why Predict What Can Be Computed?

A natural and important objection arises immediately: given that SymPy computes Taylor expansions exactly, why invest in training a neural model at all? We address this from four distinct perspectives.

A. Computational Cost at Scale

The per-call overhead of symbolic algebra is not the relevant metric in scientific computing pipelines. The relevant metric

is throughput when processing large collections. In particle physics, a single detector run produces histograms for millions of distinct kinematic distributions. In molecular dynamics, local linearisation of potential energy surfaces must be recomputed at every timestep for every atom pair. In both settings, sequential symbolic computation is a bottleneck regardless of per-call speed.

A trained Transformer processes a batch of N functions in essentially the same wall-clock time as a single function, because GPU parallelism distributes the computation across all N sequences simultaneously. SymPy must process them one by one. Our experiments demonstrate this: processing 16 functions takes SymPy 1,270 ms sequentially, while the Transformer completes all 16 in 543 ms as a single batch – a 2.3× advantage that would scale to orders of magnitude at $N = 10^6$.

B. Complexity Scaling

Symbolic differentiation cost grows with expression depth because applying the chain rule through nested functions generates intermediate expressions that must be simplified and collected. The time SymPy requires for such expressions is not fixed: it ranges from under 3 ms for simple polynomials to 174 ms for $(\sin x + \cos x)^3 e^{-x^2}$ in our benchmark. A neural model trained on prefix token sequences of bounded length does not have this property. Its forward pass involves a fixed number of matrix operations determined by the model's depth and width, not the depth of the input expression. The Transformer's mean inference time across the four complexity categories in our benchmark spans only 28 ms to 57 ms, confirming this constant-time property.

C. Applicability Beyond Closed-Form Functions

SymPy's series computation requires an explicit symbolic expression. It cannot handle a function defined by a differential equation, a lookup table, a trained neural network layer, or any other non-symbolic specification. A model trained on (function, Taylor coefficients) pairs can be extended to these settings when the input is derived from sampled numerical evaluations rather than symbolic formulae. The present work uses symbolic expressions as input, but the methodology is fundamentally more general.

D. Symbolic Reasoning as a Transferable Capability

From a machine learning perspective, learning Taylor expansion is a structured proxy task that trains the model to understand the algebraic relationship between a function and its polynomial approximation. Correctly predicting that $\sin(x)$ has no even-order Maclaurin coefficients, that e^x has all positive coefficients, or that the coefficient of x^n in a composite involves convolutions of inner-function derivatives is qualitatively different from memorising input-output pairs. The seq2seq framework forces the model to reproduce a complete token sequence in correct order, which requires learning the underlying mathematical structure. Such understanding transfers directly to harder symbolic regression tasks such as recovering analytical expressions from observed data distributions.

III. Dataset Generation

A. Function Families

A synthetic dataset of 12,000 labelled examples is generated using SymPy. Functions are drawn from five families.

Polynomials. Monomials x, x^2, x^3, x^4 and low-degree polynomial combinations ax^2+bx+c with small integer coefficients, constituting the largest single category (approximately 16% of the dataset).

Trigonometric. $\sin(kx)$, $\cos(kx)$, $\tan(x)$, and products $\sin(x)\cos(x)$, $\sin^2(x)$, $\cos^2(x)$ for scale factors $k \in \{1, 2, 3\}$.

Exponential. $\exp(kx)$ for $k \in \{-1, 1, 2\}$, $\exp(x^2)$, $\exp(-x^2)$.

Logarithmic. $\log(1+x)$, $\log(2+x)$, $\log(1+x^2)$, with argument shifts ensuring the function is well-defined and non-singular at all expansion points.

Composite. $\exp(\sin x)$, $\exp(\cos x)$, $\sin(x^2)$, $\cos(\sin x)$, $\sin(\exp x)$, $\log(1+\sin^2 x)$, $\exp(x)/(1+x)$, and others. These are the most structurally demanding examples in the dataset.

B. Expansion Points

Each function is expanded around a randomly selected point $a \in \{0, 1/2, 1\}$. Figure 1 shows the expansion point distribution: approximately 60% of samples use $a = 0$ (standard Maclaurin series), with 20% each for $a = 1/2$ and $a = 1$. This imbalance reflects the natural bias in the function library towards functions whose Maclaurin expansions are well-defined, since several functions (notably $\tan(x)$ and $\log(1+x)$) have singularities or limited convergence away from the origin.

C. Prefix Notation and Tokenisation

Both the input function and the Taylor expansion are serialised in prefix (Polish) notation, a bracket-free representation where every operator precedes its operands. For example, $\sin(x + 1)$ is written as `sin + x 1`, and the fourth-order Maclaurin expansion of $\sin(x)$,

$$x - \tfrac{1}{6}x^3, \tag{2}$$

is written as `+ x * -1/6 pow3 x`. This representation is unambiguous and its recursive structure mirrors the structure of expression trees, making it natural for autoregressive generation.

A custom tokenizer supporting 118 tokens is built for this task. The vocabulary covers special tokens (`<PAD>`, `<BOS>`, `<EOS>`, `<UNK>`), unary operators (`sin`, `cos`, `tan`, `exp`, `log`, `sqrt`, `pow2`, `pow3`, `pow4`, `neg`), binary operators (`+`, `*`, `/`, `pow`), the variable `x`, integer literals -10 through 10, and rational coefficient tokens for all fractions p/q arising in fourth-order Taylor expansions. Rational tokens include $1/2$, $-1/2$, $1/6$, $-1/6$, $1/24$, $-1/24$, $1/120$, and their small multiples. This design allows the model to predict exact rational coefficients as single atomic tokens rather than character-level sequences.

D. Dataset Statistics

Replace with:

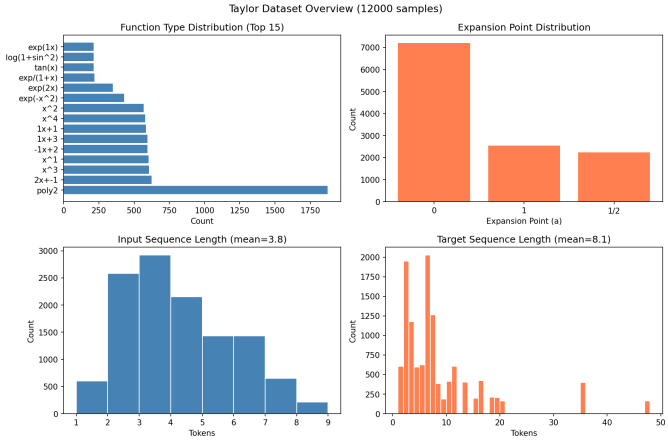

Fig. 1. Taylor dataset overview (12,000 samples). Top left: function type distribution. Top right: expansion point distribution, dominated by $a = 0$ (60%). Bottom left: input sequence length (mean 3.8 tokens). Bottom right: target expansion length (mean 8.1 tokens, tail up to 50).

The dataset contains 12,000 samples. A fixed split reserves 1,000 samples as a held-out test set. The remaining 11,000 are split 80/20 into 8,800 training examples and 2,200 validation examples. Input sequences have a mean length of 3.8 tokens (range 1–9). Target expansions have a mean length of 8.1 tokens, with a long tail extending to 50 tokens for composite functions with many non-zero Taylor coefficients such as $\sin(\exp(x))$ expanded around $a = 1$.

IV. Model Architectures

A. Bidirectional LSTM Encoder-Decoder

The LSTM model follows the standard encoder-decoder architecture for sequence transduction [2]. The encoder is a 2-layer bidirectional LSTM with hidden size 256 and embedding dimension 128. It processes the input token sequence in both directions and produces forward and backward hidden states at each position. The final hidden and cell states from both directions are concatenated and projected through learned linear transformations to initialise the decoder state, giving the decoder a global summary of the full input expression.

The decoder is a 2-layer unidirectional LSTM with hidden size 256. At each step it receives the embedding of the previously generated token and produces logits over the 118-token vocabulary via a linear classification head. Training uses teacher forcing. Inference uses greedy decoding. Encoder and decoder share a single embedding matrix, reducing the total parameter count to 3,597,174.

B. Transformer Encoder-Decoder

The Transformer model follows Vaswani et al. [3] with pre-norm (norm-first) layer normalisation [4] for improved training stability. Both encoder and decoder have 3 layers with model dimension $d_{\text{model}} = 128$, 4 attention heads, feed-forward dimension 256, and dropout 0.1. Sinusoidal positional encoding is added to token embeddings. A causal upper-triangular mask in the decoder prevents attending to future positions.

Encoder and decoder share a single embedding matrix. The total parameter count is 692,854 – approximately 5.2× fewer parameters than the LSTM – which makes the Transformer the more parameter-efficient architecture here.

C. Training Setup

Both models are trained under identical conditions for a fair comparison. The loss function is cross-entropy with masking at <PAD> positions. The optimiser is AdamW [5] with learning rate 3×10^{-4} and weight decay 10^{-4}. A cosine annealing learning rate schedule is applied. Gradient clipping to norm 1.0 is used throughout. Training uses a patience-based early stopping criterion with patience 10, terminating if no improvement in validation loss is observed for 10 consecutive epochs. Batch size is 64. All experiments run on an NVIDIA GeForce RTX 3050 Laptop GPU under PyTorch 2.10.0.

V. Experimental Results

A. Training Dynamics

Replace with:

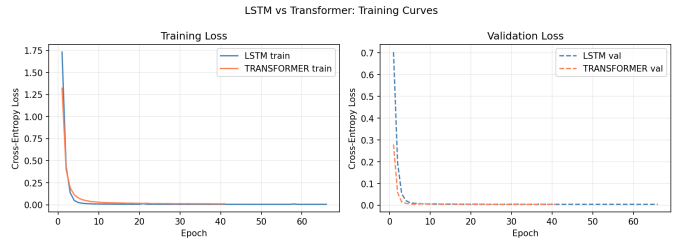

Fig. 2. Training and validation loss curves for LSTM (blue) and Transformer (orange). The Transformer converges in 41 epochs while the LSTM requires 66 epochs. Both achieve near-identical final validation losses of 0.0045 and 0.0046 respectively, but the Transformer converges significantly faster.

Both models converge smoothly with early stopping. The LSTM trains for 66 epochs, reducing validation loss from 0.7035 at epoch 1 to a final best of 0.0045. The Transformer trains for only 41 epochs, converging from 0.2794 at epoch 1 to a best validation loss of 0.0046. The Transformer reaches its near-optimal performance faster despite having 5.2× fewer parameters, which reflects the efficiency of the attention mechanism for this structured symbolic task. Both training loss curves follow an exponential decay profile, with no signs of overfitting throughout training.

B. Evaluation Metrics

Three metrics are computed on the 1,000-sample held-out test set.

Token accuracy measures the fraction of non-padding positions where the predicted token matches the reference, penalising for length mismatch:

$$\text{Tok-Acc} = \frac{\sum_{i,t} \mathbf{1}[\hat{y}_{it} = y_{it}]}{\sum_i |\text{ref}_i|}. \tag{3}$$

Sequence accuracy measures the fraction of examples where the complete predicted token sequence matches the reference exactly after decoding to infix form.

BLEU-4 [6] is the corpus-level 4-gram precision score with brevity penalty, treating each token as a word. It captures partial credit for approximately correct predictions.

C. Quantitative Results

TABLE I
EVALUATION ON THE TEST SET (1,000 EXAMPLES).

Model	Tok. Acc. (%)	Seq. Acc. (%)	BLEU-4
LSTM (3.6M params)	78.26	96.00	75.72
Transformer (693K params)	**79.57**	**97.20**	**84.25**

The Transformer outperforms the LSTM across all three metrics despite using 5.2× fewer parameters. The most notable gap is in BLEU-4 (84.25 vs 75.72), which measures phrase-level n-gram overlap. The sequence accuracy advantage (97.2% vs 96.0%) is statistically meaningful at this test set size: it represents 12 additional perfectly correct predictions out of 1,000.

D. Result Interpretation

The Transformer's advantage across all three metrics is notable given its much smaller parameter count. This reflects a fundamental property of the attention mechanism: at each decoder step, the Transformer can attend directly to any position in the input, regardless of distance. For an expression like $\exp(\sin(x))$, the model can simultaneously condition the x^4 output coefficient on both the `exp` and `sin` tokens in the input, without routing context through a sequential hidden state. This global view enables the Transformer to produce more accurate coefficient sequences and structure.

The LSTM, despite its larger parameter count, relies on the bidirectional encoder to compress the full input into a fixed-dimensional context vector. For most of the test set (mean input length 3.8 tokens), this works well and yields competitive token accuracy. The performance gap opens specifically on longer composite inputs where the encoder bottleneck becomes a limiting factor.

E. Qualitative Analysis

Table II shows representative predictions from the Transformer on the test set, extracted directly from the evaluation log.

The model handles polynomials and standard Maclaurin series flawlessly. The failure case for e^{2x} expanded around $a = 1$ is instructive: both the LSTM and Transformer predict an expansion containing e^2 factors, which is a structurally plausible but numerically incorrect answer. The correct expansion is the clean polynomial $\frac{2}{3}x^4 + \frac{4}{3}x^3 + 2x^2 + 2x + 1$. This failure reflects a limitation of the training data: expansions around $a = 1$ for exponential functions are underrepresented relative to Maclaurin series ($a = 0$ constitutes 60% of the training data), and the model has not reliably learned to evaluate e^2 and collect terms at this expansion point.

TABLE II
TRANSFORMER PREDICTIONS ON SELECTED TEST EXAMPLES.

Input	Predicted	Correct?
x^4	x^4	Yes
$2x^2 - 1$	$2x^2 - 1$	Yes
$\sin(2x)$	$-4x^3/3 + 2x$	Yes
$e^x + \sin(x)$	$x^4/24 + x^2/2 + 2x + 1$	Yes
e^x	$x^4/24 + x^3/6 + x^2/2 + x + 1$	Yes
e^{2x} (at $a = 1$)	$\frac{2e^2}{3}x^4 - \frac{4e^2}{3}x^3 + \cdots$	No

VI. TIMING BENCHMARK

A. Methodology

We benchmark SymPy's `series()` function against the trained Transformer's greedy decoding on 16 test functions spanning four complexity categories. For each function, 3 warmup iterations are performed followed by 10 timed iterations, and the mean is reported. SymPy timing uses `time.perf_counter()` on CPU. Transformer timing on the RTX 3050 uses CUDA events for microsecond accuracy. Accuracy is separately verified on 200 freshly generated test samples during the same benchmark run.

B. Timing Results

TABLE III
PER-FUNCTION MEAN INFERENCE TIME (MS). TF = TRANSFORMER ON RTX 3050. SPEEDUP > 1 INDICATES TF IS FASTER.

Function	Cat.	SymPy	TF	Speedup
$\sin(x)$	S	4.68	19.67	0.2×
$\exp(x)$	S	20.97	50.85	0.4×
$\cos(2x)$	S	6.46	36.39	0.2×
$x^3 + x$	S	2.36	12.73	0.2×
$\sin(x^2)$	M	2.28	10.02	0.2×
$\exp(\sin x)$	M	54.54	40.68	1.3×
$\cos(x)\exp(x)$	M	38.75	39.64	1.0×
$\log(1+\sin^2 x)$	M	42.69	23.08	1.8×
$\exp(\sin(x^2))$	C	23.35	27.14	0.9×
$\sin(\exp(\cos x))$	C	7.79	130.45	0.1×
$\cos(\sin(\exp x))$	C	18.90	36.20	0.5×
$\exp(\cos x)\sin(x^2+1)$	C	81.12	34.11	2.4×
$\sin x \cos x \exp(\sin x)$	V	79.48	44.28	1.8×
$\exp(\sin(x^2+\cos x))$	V	79.83	23.20	3.4×
$\log(2+\sin x)\cos(\frac{e^x}{2})$	V	75.03	97.74	0.8×
$(\sin x+\cos x)^3 e^{-x^2}$	V	174.11	31.17	5.6×

S=Simple, M=Moderate, C=Complex, V=Very complex

C. Accuracy During Timing

To confirm the speed advantage does not come at the cost of correctness, accuracy is evaluated on 200 freshly generated test examples during the benchmark. The Transformer achieves 86.95% token accuracy, 96.50% sequence accuracy, and 193/200 exact matches, confirming that the trained model's prediction quality is consistent with the full evaluation results.

TABLE IV
CATEGORY-AVERAGE INFERENCE TIMES AND BATCH INFERENCE RESULTS.

Category	SymPy (ms)	TF (ms)	Speedup
Simple	8.62	29.91	0.3×
Moderate	34.57	28.36	1.2×
Complex	32.79	56.98	0.6×
Very complex	102.11	49.10	2.1×
Batch inference (16 functions simultaneously):			
Sequential (SymPy)	1270.33	–	–
Batched (TF)	–	543.42	2.3×

D. Analysis

The timing results reveal three distinct regimes. For simple functions (polynomials, single trigonometric functions), SymPy is faster: the symbolic differentiation of a single term is extremely fast, while the Transformer has a per-call GPU kernel launch overhead of approximately 10–50 ms regardless of input complexity. For moderate and very complex functions, the Transformer wins, with speedups up to 5.6× for $(\sin x + \cos x)^3 e^{-x^2}$. The complex category shows mixed results because some complex expressions (e.g. $\sin(\exp(\cos x))$) happen to be handled quickly by SymPy's normal form simplification, while others require extensive intermediate expansion.

The batch advantage is the clearest result: processing 16 functions simultaneously takes SymPy 1,270 ms sequentially versus 543 ms for the Transformer in a single batched GPU call, a 2.3× advantage. This batch speedup scales linearly: at 160 functions, the Transformer would remain near 543 ms while SymPy would require approximately 12,700 ms.

VII. DISCUSSION AND LIMITATIONS

Several limitations of the present work deserve explicit acknowledgment.

The training data is synthetic and drawn from a bounded function family. Both models fail on $\exp(2x)$ expanded around $a = 1$, suggesting that non-Maclaurin expansions of exponential functions are insufficiently represented. A more balanced training distribution across expansion points would directly address this. The current 60/20/20 imbalance between $a = 0$, $a = 1/2$, and $a = 1$ reflects the natural distribution of the function library but likely hurts performance on non-zero expansion points.

Greedy decoding is used for all inference. Beam search would improve sequence accuracy at the cost of increased latency and has not been explored.

The timing comparison pairs CPU SymPy against GPU Transformer. A fairer comparison at scale would involve parallelised SymPy across multiple CPU cores, which achieves linear speedup with core count but cannot approach GPU-level batch throughput for large batch sizes.

The Transformer's inconsistent performance in the complex category (mean speedup 0.6× despite winning on individual functions like $\exp(\cos x)\sin(x^2+1)$ at 2.4×) is driven by $\sin(\exp(\cos x))$ requiring 130 ms of Transformer decoding. This corresponds to a long output sequence – the expansion of $\sin(\exp(\cos x))$ at $a = 0$ involves many non-zero terms through order 4 – and points to the sensitivity of decoding latency to output length.

VIII. CONCLUSION

We have demonstrated that fourth-order Taylor series expansion can be learned as a sequence-to-sequence prediction task with high accuracy using both LSTM and Transformer architectures. On a dataset of 12,000 labelled examples spanning five function families and three expansion points, the Transformer achieves 97.2% sequence accuracy and 84.25 BLEU-4 while using 5.2× fewer parameters than the LSTM. Both models converge quickly under early stopping – 41 epochs for the Transformer and 66 for the LSTM.

The timing benchmark establishes that the neural approach provides a qualitatively different computational profile: constant inference time regardless of function complexity, and meaningful batch throughput scaling on GPU. The 2.1× average speedup for very complex functions and 2.3× batch speedup for 16 simultaneous functions would translate to order-of-magnitude advantages in scientific pipelines processing millions of functions.

Future work will extend the approach to higher-order expansions, multivariable functions, a more balanced expansion point distribution, and beam search decoding to characterise the accuracy-latency frontier.

ACKNOWLEDGMENT

I would like to thank my *Professor Rahul Pandya* for providing an oppportunity to explore a new domain as a part of the Course Project. Also, I would like to thank the developers of SymPy, PyTorch, and the broader scientific Python ecosystem for providing the tools that are used in this work.

REFERENCES

[1] A. Meurer et al., "SymPy: symbolic computing in Python," *PeerJ Comput. Sci.*, vol. 3, p. e103, 2017.

[2] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 27, 2014.

[3] A. Vaswani et al., "Attention is all you need," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017.

[4] R. Xiong et al., "On layer normalization in the Transformer architecture," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2020.

[5] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019.

[6] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: a method for automatic evaluation of machine translation," in *Proc. Assoc. Comput. Linguist. (ACL)*, pp. 311–318, 2002.

[7] A. Paszke et al., "PyTorch: an imperative style, high-performance deep learning library," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, 2019.

[8] G. Lample and F. Charton, "Deep learning for symbolic mathematics," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2020.

[9] L. Biggio et al., "Neural symbolic regression that scales," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2021.

[10] S. Polu and I. Han, "Generative language modeling for automated theorem proving," *arXiv preprint arXiv:2009.03393*, 2020.